Implement a Rate Limiting Interceptor

Objective

Develop a rate limiting feature to manage request counts and enforce limits in a concurrent environment. The rate limiting should be implemented as an Interceptor in a NestJs application. You can use anything you want (Cursor, ChatGPT, and so on…)

Backend Requirements

1. Storage: Consider using external storage/cache to store and manage the request count for each user (based on user id, and not IP/MAC).

2. Framework: Use the **NestJS** framework for implementing this task:
npm i -g @nestjs/cli
nestjs new project-name
cd project-name

```
@Injectable()
export class RateLimitInterceptor implements NestInterceptor {
  private readonly redisClient = Redis.createClient();
  async intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
  }
}
```

3. Rate Limiting Middleware: Implement middleware that:
- Limits each user to 100 requests per hour.
- Uses external components to track the number of requests per user.
- Resets the count every hour.

4. X-count Header:
- In each successful request add a header of x-count and how many requests are left.

5. Concurrency Handling:
- Ensure that the rate limiting mechanism is concurrency-proof, handling multiple requests simultaneously without letting any slip through the cracks.

6. Error Handling:
- Return a 429 (Too Many Requests) HTTP status code when a user exceeds the rate limit.
- Include a meaningful error message and the time when the user can try again.

7. Testing:
- Write unit tests for the rate limiting middleware.
- **Use a mock library for your external storage/cache service.**

Frontend Requirements

Use the latest React/Angular with typescript and with your favorite UI library.

Core Features

1. **Live User Table**
    - Displays: User ID, Remaining / Total, Reset Time, Status (OK / Warning / Exceeded), Last Request.
    - **Updates live via WebSocket (please use @nestjs/websockets)** (`rate:update`, `rate:limit_exceeded`)
2. **User Detail View**
    - Shows current stats, recent requests, and last reset time.
    - Real-time event stream for that user.
    - Option to view short activity history (past hour/day).
3. **Admin Controls**
    - **Reset limit** for a user.
    - **Adjust limit** (custom value).
    - **Suspend or unlock** throttling temporarily.
    - **Add extra requests** within current window.
4. **Search & Filters**
    - Search by user ID.

    - Filter by rate limit (Exceeded = 100%, Warning >= 90%, OK <= 90%) or last active time.

5. **Analytics**

    - Summary cards: total users, % near limit, 429 count today.

    - Top users by requests in the current window.

Submission Guidelines

- The code should be submitted as a zip file containing all the necessary files, or a link to a GitHub repository.
- Include a `package.json` file with all the necessary dependencies and scripts.
- Ensure the code is well-commented and follows good coding practices.